Panel Viva Preparation - Anticipated Questions & Answers

Intent-Aware Deception System for Real-Time Attacker Profiling – generated from docs/PANEL_VIVA_PREP.md

---

INT-AWARE -- Panel Viva Preparation Pack

Project: Intent-Aware Deception System for Real-Time Attacker Profiling **Deployed model:** `linsvc_calibrated` - **94.49 %** held-out accuracy - **0.9229** macro-F1 **Every number in this pack is real** and traceable to `backend/app/ml/artifacts/metrics.json`.

> **How to use this pack.** Section 1 answers the question the panel already asked you and you could not answer. Learn that one properly -- it is the question most likely to be repeated. Sections 2-12 are the questions an expert panel actually asks, grouped by theme, each with a short answer you can say out loud in 20-40 seconds. Section 13 is your "if you don't know" script. Section 14 is the numbers cheat-sheet -- memorise it.

---

1. THE SECTOR QUESTION (the one you were asked last time)

> **"Focus on one sector -- finance, healthcare, whatever. Where would you actually implement this?"**

1.1 Your answer: BFSI (Banking, Financial Services & Insurance)

Say this:

> "We are positioning INT-AWARE for the **BFSI sector**, specifically to protect the **SSH/Telnet administrative layer of a bank's server infrastructure** -- the jump hosts, bastion servers, and internal Linux fleet. We chose BFSI for three reasons: it is the most attacked sector, it is the most heavily regulated, and -- most importantly -- it is the one sector where our system's **explainability** is not a nice-to-have but a **compliance requirement**."

Then give the three reasons in order:

Reason 1 -- The threat profile matches our taxonomy exactly. Attacks on banking infrastructure follow the exact kill chain we model: credential access -> privilege escalation -> persistence -> **exfiltration** -> log cleanup. Our risk engine already weights `EXFILTRATION` highest (35) and `CLEANUP` as an evasion signal -- that weighting *is* a BFSI threat model. A bank's crown jewel is data exfiltration, and that is precisely the intent we score most severely.

Reason 2 -- Explainability is a regulatory requirement, not a feature. This is your strongest point. In BFSI, a security control that cannot explain its decision is very hard to get past an auditor. INT-AWARE gives, for every single event, the **deciding source** (RULE / ML / HYBRID / GEMINI / FALLBACK) and a **calibrated confidence**, and every risk score decomposes into transparent additive terms. That maps onto:

Requirement	How INT-AWARE satisfies it
RBI Cyber Security Framework (Indian banks) -- requires a SOC, attack detection, and forensic readiness	Per-session attacker profiles with full intent trajectory and timestamps = ready-made forensic record
PCI-DSS Req. 10 -- track and monitor all access	Every captured event is persisted immutably with source IP, session, command, intent
PCI-DSS Req. 11.5 -- deploy intrusion-detection techniques	The honeypot + hybrid classifier is a detection control with zero false-positive cost (see 1.3)
DPDP Act 2023 / GDPR breach notification -- you must report *what* was accessed within a deadline	Intent classification tells you *what the attacker was trying to do*, not just that they connected -- this shortens breach triage dramatically

Reason 3 -- Zero false positives on a honeypot, which is what a bank SOC needs. A bank SOC is drowning in alerts. A honeypot has a unique property: **no legitimate user has any reason to touch it**, so every single interaction is suspicious by definition. That means our alerts are inherently high-fidelity -- we are not adding noise to an already-noisy SOC, we are adding a **high-signal, low-volume** feed.

1.2 Where exactly it sits in a bank's network (be concrete -- they will ask)

Draw or describe this:

```
Internet ■ [ DMZ / Perimeter Firewall ] ■ ■■■■ Real bastion / jump host (port 22, hardened, real admins) ■ ■■■■ INT-AWARE honeypot (port 22 on a decoy IP, or 2222) ■ NO production data, NO real credentials ■ Isolated VLAN, egress-blocked ▼ Intent classification + risk scoring ▼ Feed to the bank's SIEM (Splunk / QRadar / Sentinel) ▼ SOC analyst dashboard + auto-alert at risk >= 75
```

Key deployment points to state: - It is deployed as a **decoy alongside** the real bastion host, on the same subnet so it looks legitimate to an attacker doing internal reconnaissance. - It holds **no production data and no real credentials** -- nothing of value is at risk. - It sits on an **isolated VLAN with egress blocked**, so it cannot be used as a pivot. - Output is pushed to the bank's **existing SIEM**, so it augments rather than replaces their stack.

1.3 The business value in one line (memorise this)

> "It turns the honeypot from a passive log collector into an **early-warning system that tells the SOC an attacker's objective before they reach production** -- and because a honeypot has no legitimate users, it does that with essentially zero false positives."

Concrete BFSI use case to quote: An attacker compromises a vendor's VPN credentials and starts internal reconnaissance. They hit our decoy bastion. Within seconds INT-AWARE classifies `RECONNAISSANCE -> DISCOVERY -> CREDENTIAL_ACCESS`, the session risk crosses 75, and the SOC gets a **CRITICAL alert naming the source IP and the attacker's inferred objective** -- while the attacker is still probing the decoy and has not yet touched a production system. That is dwell-time reduction, which is the metric BFSI security teams are measured on.

1.4 Your backup answer if a panelist pushes a different sector

Do not fight them. Say: *"The architecture is sector-agnostic; what changes is the risk weighting and the deployment point. If you'd like a healthcare framing:"*

Sector	Deployment point	What you re-tune	Driving regulation
BFSI (our primary)	Decoy bastion beside real jump hosts	`EXFILTRATION` weighted highest	RBI framework, PCI-DSS, DPDP
Healthcare	Decoy beside HIS/PACS servers; legacy medical devices can't be patched, so a decoy is the practical control	Weight `PERSISTENCE` + `EXECUTION` higher -- ransomware pre-staging is the threat, not theft	HIPAA, DPDP Act
Critical infra / OT	Decoy in the IT/OT DMZ	Weight `DISCOVERY` highest -- OT recon precedes physical impact	IEC 62443, NCIIPC guidelines
Cloud / SaaS	Decoy container or SSH-exposed VM per tenant	Weight `PRIVILEGE_ESCALATION` -- container escape is the risk	ISO 27001, SOC 2

The meta-point that impresses panels: *"The taxonomy and the pipeline stay identical across sectors -- only the risk weight table changes, and that is a single static dictionary in `risk_engine.py`. That is deliberate: sector adaptation is configuration, not re-engineering."*

1.5 If they ask "why not just pick healthcare, isn't it more novel?"

> "Healthcare is a strong second and our architecture supports it. We led with BFSI because the **auditability** of our system is its most distinctive property, and BFSI is the sector where auditability is legally mandated and therefore commercially valued. In healthcare the dominant threat is ransomware, which is more about *execution and persistence* than *intent diversity* -- so it exercises less of what makes our system novel. We optimised the framing for where our contribution matters most."

---

2. NOVELTY & MOTIVATION

Q: What is actually new here? Cowrie already exists.

> Cowrie is a **sensor** -- it captures and logs. It tells you *what* was typed. It has no notion of *why*. Our contribution is the **interpretation layer** on top: a 10-class MITRE-aligned intent taxonomy, a hybrid arbitration architecture that fuses rules + ML + an optional LLM by explicit confidence precedence, an explainable additive risk engine, and multi-stage trajectory assembly -- all in real time. We did not rebuild the

> honeypot; we made it *understand* what it captures.

Q: So your novelty is just applying an SVM to command logs?

> No -- the SVM is a component, not the contribution. The contribution is the **arbitration architecture**: deterministic rules give precision and auditability on known signatures; the calibrated ML gives generalisation and a probabilistic confidence; the optional LLM gives semantic reach on genuinely novel input. Each covers the others' failure mode. Crucially ML is **mandatory** and the LLM is **optional** -- the system runs fully with no API key -- so we get semantic reach without a hard dependency on a non-deterministic, paid, latency-adding external service.

Q: Why is inferring intent better than just alerting on a signature?

> A signature fires on a command. An intent tells you an **objective**, and objectives compose into a trajectory. `cat /etc/shadow` alone is one alert. `cat /etc/shadow` -> `wget payload.sh` -> `rm auth.log` is *credential access -> execution -> evasion* -- a coherent, escalating attack that deserves a far higher score than three isolated alerts. We score the **session**, not the command.

Q: Who is the end user?

> A SOC analyst (Tier 1/Tier 2) in a bank. They open the dashboard, see ranked sessions by risk, and for each one immediately see the attacker's inferred objective, the MITRE tactics touched, and why the system decided that. It shortens triage.

---

3. THE DATASET -- YOUR BIGGEST ATTACK SURFACE

Experts *will* hit this. Do not be defensive; be precise and pre-emptive.

Q: Your dataset is synthetic/generated. Isn't that a fatal weakness?

> It is our main limitation and we report it openly. Three things to note. First, it is not random -- it is expanded from **242 curated command template families** built from real attacker tradecraft and MITRE technique descriptions, so the *commands* are realistic even though the *corpus* is generated. Second, labels are correct **by construction** -- each command inherits its family's intent -- so we have zero label noise, which a hand-labelled harvested corpus would not. Third, and most importantly, we **quantified** the cost of this choice: our leave-families-out experiment holds out whole templates and scores **67.87 %**. We publish that number rather than hiding behind the 94 %.

Q: Then your 94.49 % is meaningless.

> The 94.49 % is a valid measure of one thing: how well the model classifies commands drawn from the same distribution, with **leakage control asserted in code** -- no identical command string appears in both train and test. That is why it is 94 % and not 100 %. The 67.87 % measures a different and harder thing: generalisation to command shapes never seen. Both are real, both are reported, and they answer different questions. Quoting only one would be dishonest.

Q: How would you fix the synthetic-data limitation?

> Deploy the honeypot on a public IP and harvest real traffic -- which the running system already supports, since it ingests live Cowrie telemetry through the same pipeline. Then retrain on harvested commands with human-verified labels, and re-run the identical leave-families-out protocol to measure whether the gap closes. That is our stated future work, and it needs no architectural change.

Q: 1360 samples is small.

> Agreed for a deep model; adequate for a **linear model on sparse high-dimensional TF-IDF features**, which is exactly why we chose that model class. With 1360 samples a transformer would overfit badly. Our 4-fold CV std is tight (**+/-0.0181**), which indicates the estimate is stable rather than a lucky split.

Q: Your classes are imbalanced -- EXECUTION has 318, PRIVILEGE_ESCALATION has 57.

> Yes, and we handle it three ways. We use `class_weight="balanced"` in the LinearSVC so minority classes are not drowned out. We report **macro-F1 (0.9229)** as our headline, not accuracy, because macro-F1 weights every class equally regardless of size. And we use **stratified** splitting and stratified CV so proportions are preserved. The imbalance is also not arbitrary -- it reflects that real attackers issue far more execution commands than privilege-escalation commands.

Q: How did you avoid data leakage?

> An explicit assertion in the training code that the intersection of train and test command strings is empty. Plus the `family_id` group key enables the stricter leave-families-out split where no *template* is shared, not just no exact string.

---

4. MACHINE LEARNING -- MODEL CHOICE

Q: Why LinearSVC and not deep learning / BERT / an LLM?

> Four reasons, in order of importance. **(1) Data size** -- 1360 samples cannot support a transformer without severe overfitting. **(2) Latency** -- this is a real-time path; a linear model over sparse features classifies in well under a millisecond, a transformer does not. **(3) Explainability** -- a linear model's decision traces to weighted n-gram features; we can point to the token that drove the decision, which matters for our audit story. **(4) We measured it** -- we compared three candidates by 4-fold CV and the calibrated LinearSVC won at **0.8539** vs LogisticRegression 0.8482 vs ComplementNB 0.8146. We also evaluated a RandomForest during development and dropped it for poor generalisation and poor calibration. We chose empirically, not by assumption.

Q: Why TF-IDF, and why both word and character n-grams?

> This is the single most important design choice for the shell-command domain. The **word** vectoriser uses a whitespace tokeniser `(?u)\S+` deliberately -- standard tokenisers shatter `/etc/shadow` and `-O` into meaningless fragments, while ours keeps flags, paths, and URLs intact as single tokens. The **character** vectoriser (`char_wb`, 3-6 grams) catches sub-token cues a word model structurally cannot: flag fragments, path fragments, and obfuscation. Attackers vary spelling and inject noise; character n-grams are robust to that. We union both, so the model sees whole tokens *and* their internal structure.

Q: What is calibration and why do you need it?

> A raw SVM outputs a **margin**, which is a distance, not a probability -- a margin of 2.0 does not mean 66 % confident. We wrap it in `CalibratedClassifierCV(method="sigmoid")`, which is **Platt scaling**: it fits a sigmoid mapping margins to true probabilities. We need this because our hybrid layer **thresholds on confidence** (accept ML at >= 0.40). Thresholding an uncalibrated margin would be meaningless. Calibration is what makes the arbitration logic principled rather than arbitrary.

Q: Why is your ML acceptance threshold 0.40? That seems low.

> Because it is not a standalone decision threshold -- it sits *inside* a precedence chain. A strong rule (>= 0.90) already outranks ML, so ML only speaks when no confident rule fired. In a 10-class problem, chance is 0.10, so 0.40 is 4x chance and a meaningful signal. And if ML is below 0.40 we do not guess -- we fall through to the LLM, then to best-available, then explicitly to `OTHER` with source `FALLBACK`. The threshold is tuned for the chain it lives in, and the system is honest when nothing is confident.

Q: Your weakest class is RECONNAISSANCE at F1 0.714. Why, and does it matter?

> Why: short probes like `whoami`, `id`, `uname` are lexically almost identical to DISCOVERY and CREDENTIAL_ACCESS probes, and it is our smallest test class at support 16, so each error costs ~6 points of recall. Does it matter: **less than the number suggests**, for two reasons. First, the confusion matrix shows errors stay within **neighbouring** intents -- reconnaissance confused with discovery -- and never cross into an unrelated tactic, so the kill-chain interpretation and the risk tier barely change. Second, this exact ambiguity is what the **rule layer** resolves at runtime: `whoami` is a deterministic high-confidence rule match, so in the live system it never depends on the ML model at all.

Q: How do you know you're not overfitting?

> Four controls. Held-out test set never touched during model selection. Model selection done by CV **on the training split only**. Leakage assertion in code. And a tight CV standard deviation (+/- 0.0181) indicating stability across folds. The leave-families-out score also acts as a deliberate stress test -- an overfit model would collapse far below 67.87 %.

Q: Why macro-F1 as the headline instead of accuracy?

> Because with imbalanced classes accuracy flatters you -- a model could score well by nailing EXECUTION (318 samples) and failing PRIVILEGE_ESCALATION (57). Macro-F1 averages per-class F1 with equal weight, so a small class failing is fully visible. It is the honest metric for our distribution, which is why we also selected the model on CV macro-F1.

Q: Is it reproducible?

> Yes, deterministically. Fixed `RANDOM_SEED = 1337` across dataset generation, splitting, CV, and training; pinned versions (Python 3.13.12, scikit-learn 1.9.0, numpy 2.5.2); and all metrics are regenerated by re-running the training script -- nothing in our figures or tables is hand-typed.

5. HYBRID ARCHITECTURE

Q: Walk me through the arbitration order.

> Five steps, strict precedence. **(1)** Strong rule -- confidence >= 0.90 and label ≠ OTHER -> authoritative, source `RULE`. **(2)** Confident ML -- calibrated probability >= 0.40 -> source `ML`, or `HYBRID` if the rule independently agrees. **(3)** Gemini -- consulted **only** if 1 and 2 were both inconclusive -> source `GEMINI`. **(4)** Best-available -- highest-confidence non-OTHER among rule and ML. **(5)** Fallback -- nothing above 0.10 -> `OTHER`, source `FALLBACK`. Every event records which step decided it.

Q: Why do rules outrank ML? Isn't ML supposed to be better?

> For **known signatures**, a curated rule is strictly better: it is 100 % precise on what it matches, instant, and fully auditable. ML's advantage is generalising to the unknown. So we let each do what it is best at -- rules handle the known, ML handles the unseen. Our rule layer is 13 rules over 43 curated regex patterns, each with an attached confidence.

Q: What if the rule and the ML disagree?

> Precedence decides, and we record the disagreement. If the rule is strong (>= 0.90), the rule wins and source is `RULE`. If the rule is weak and ML is confident, ML wins and source is `ML` -- notably **not** `HYBRID`, because we reserve `HYBRID` for genuine agreement. So an analyst can distinguish "both agreed" from "ML overrode a weak rule", which is exactly the kind of signal you want when auditing a decision.

Q: Isn't depending on Gemini a weakness -- cost, latency, privacy, availability?

> That is precisely why it is **optional and last**. It is never on the hot path for known attacks -- it is consulted only when both the rule engine and the ML model are inconclusive, i.e. genuinely novel input. Its output is gated (accepted at >= 0.85) and its input sanitised. Most importantly the system **starts and runs fully without any API key** -- ML is mandatory, the LLM is not. For a bank with data-residency constraints you disable it entirely and lose only novel-command coverage, or swap in a locally hosted model behind the same interface.

Q: What is the latency of the pipeline?

> The rule and ML path is sub-millisecond per command -- regex matching plus a sparse linear dot product. The whole capture -> classify -> score -> persist path is **fully asynchronous** so ingestion never blocks, and events stream to the dashboard over WebSocket. Only the optional Gemini path adds network latency, and it is off the hot path by design.

Q: Will it scale?

> The classifier is O(non-zero features) per command and stateless, so it scales horizontally. The async collector is non-blocking. MongoDB writes are **idempotent upserts keyed on `event_id`**, so we can replay logs or run multiple collectors without creating duplicates. The realistic bottleneck is honeypot I/O, not classification. Scaling out means more honeypot instances feeding the same pipeline.

Q: What happens if a component fails -- Mongo down, model file missing?

> Model loading is **lazy and degrades gracefully**: if the artefact is absent the system still runs on rules plus the optional LLM rather than crashing. The collector is **rotation-safe** -- it detects log rotation by device/inode identity and re-opens the new file, so it never gets stuck on a stale handle. Idempotent upserts mean a restart replays safely without duplicating data.

---

6. RISK SCORING (expect "your weights are arbitrary")

Q: How is the risk score calculated?

> Additively and transparently. Base weight for the highest-severity observed intent, plus behavioural boosters from session context, clamped to [0, 100]. Weights run from OTHER = 0 and RECONNAISSANCE = 10 up to EXFILTRATION = 35. Boosters: any failed auth +5, brute-force pattern (>= 3 failed logins) +10, rapid command bursts +5, deep session (>= 5 commands) +5, high intent variety (>= 4 distinct intents) +5. Tiers: LOW 0-24, MEDIUM 25-49, HIGH 50-74, CRITICAL 75-100, and >= 75 auto-raises an alert.

Q: Those weights are arbitrary. How did you validate them?

> Answer honestly, then reframe: "They are **expert-assigned, not learned** -- that is a deliberate design decision, and it is a limitation we own. The ordering is principled: it follows kill-chain severity, so exfiltration (the attacker's objective) outweighs reconnaissance (the prelude), and the boosters capture *behaviour* -- brute forcing, breadth, depth -- that intent alone misses. The advantage of assigning rather than learning is that the score is **fully explainable and reproducible** -- we label it 'Deterministic Threat Risk' on the dashboard precisely because it is not a model estimate. A learned score would need labelled incident-outcome data we do not have, and it would sacrifice the auditability that is our main selling point in a regulated sector. Learning these weights from real incident outcomes is natural future work."

Q: Why is CLEANUP only 15 when it indicates a sophisticated attacker?

> Good challenge -- and a fair one. Our reasoning: CLEANUP alone, in isolation, could be a user clearing history, so its *base* weight is modest. But cleanup never appears alone in a real attack; it appears after execution or exfiltration, and the session is scored on its **highest-severity** intent plus an **intent-variety booster**. So a session that reaches cleanup after exfiltration already scores 35 + boosters, comfortably CRITICAL. Raising CLEANUP's base weight is a defensible tuning choice and exactly the kind of sector-specific adjustment we described -- in a bank, cleanup implies an insider covering tracks and you might weight it higher.

---

7. SECURITY, SAFETY & ETHICS

Q: Is running a honeypot dangerous? Could an attacker escape and pivot?

> The honeypot is **medium-interaction** -- it only *emulates* a shell. **No captured attacker command is ever executed** on the host or the backend. The honeypot is the sole component that touches attacker input; everything downstream treats commands as inert text to classify. In deployment it sits on an isolated VLAN with egress blocked and holds no production data or real credentials, so there is nothing of value to steal and no route to pivot.

Q: Is the LLM path an injection risk? An attacker could type a prompt.

> A real and well-spotted risk. Three mitigations: input to Gemini is **sanitised**; its output is **gated by confidence** (accepted only at >= 0.85); and its output is **constrained to our 10-class label set** -- it cannot return arbitrary text that flows into control logic, only a label we then validate. Worst case for a successful injection is one misclassified event, which is contained by the fact that risk is scored over the whole session, not one command.

Q: Legal and ethical concerns? Entrapment? Is capturing attacker activity legal?

> The system is **defensive** and should only be deployed on infrastructure you own -- we state that explicitly. It is not entrapment: the honeypot advertises no service to legitimate users and induces nothing; it passively records unauthorised access to a system the attacker had no right to touch. For our development and demos we use **RFC-5737 documentation IP ranges** (192.0.2.0/24, 198.51.100.0/24) that route nowhere, and no real malicious payloads are executed. Secrets live only in an untracked `.env`. A real BFSI deployment would additionally need legal sign-off and a data-retention policy for captured IPs, since an IP can be personal data under DPDP/GDPR.

Q: You store attacker IPs -- is that a privacy problem?

> Under DPDP/GDPR an IP can be personal data, so a production deployment needs a defined retention period and a lawful basis -- for security monitoring of your own systems that basis is normally legitimate interest. It is a compliance task, not a technical blocker, and we would flag it as a deployment prerequisite rather than claim it is already solved.

---

8. COMPARISON WITH EXISTING TOOLS

Q: Why not just use Snort / Suricata / an IDS?

> Different layer and different question. An IDS inspects **network packets** against signatures and answers "does this traffic match a known bad pattern?". We analyse **post-compromise shell behaviour** and answer "what is this attacker trying to achieve?". An IDS would not tell you that a sequence of individually-benign commands constitutes a credential-access-to-exfiltration progression. They are complementary -- we would feed our alerts into the same SIEM the IDS reports to.

Q: Why not an EDR, or SIEM correlation rules?

> An EDR protects **production** endpoints and must be extremely careful about false positives, because blocking real user activity is costly. Our honeypot has **no legitimate users**, so we can be maximally aggressive with zero business risk -- that is a structural advantage EDR cannot have. SIEM correlation rules are hand-written and brittle; our ML layer generalises to command variants nobody wrote a rule for. Again: complementary, and we integrate rather than replace.

Q: How does this compare to published honeypot-analysis research?

> Most prior work uses honeypots as **sensors** and does offline, often manual or signature-only, analysis of the resulting logs. Two gaps we close: analysis is **real-time and in-pipeline** rather than offline, and classification is **arbitrated across three heterogeneous sources with recorded provenance** rather than a single classifier. The explainability property -- every decision carries its deciding source and confidence -- is what we would emphasise as the differentiator.

---

9. IMPLEMENTATION & DEMO

Q: What is your tech stack and why?

> Backend: **Python 3.13, FastAPI, Motor** (async MongoDB) -- async throughout so ingestion never blocks. ML: **scikit-learn**. Honeypot: **Cowrie**. Frontend: **React 19 + Vite + Tailwind + Recharts**. Deployment: **Docker Compose** -- MongoDB, Cowrie, API, and dashboard start with one command. MongoDB because our events are semi-structured and schema-flexible, and we need fast aggregation for the analytics endpoints.

Q: How does the dashboard update live?

> Two WebSocket channels, `/ws/events` and `/ws/alerts`, push new events and alerts to the browser with no refresh. We additionally poll every 8 seconds as a fallback, so the counts stay correct even behind a proxy that drops WebSocket upgrades.

Q: What does the dashboard show?

> Five sections -- Overview, Live Sessions, Attacker Profiles, Threat Alerts, Analytics. Each event row carries an intent badge, a severity tier, and a **classification-source badge with the ML confidence**, which is where the explainability goal becomes visible on screen. Six analytics endpoints back the charts: intent, severity, timeline, stage, risk-distribution, and heatmap.

Q: Prove the ML is actually running live and not just evaluated offline.

> We have an end-to-end script that drives all ten intents through the **running** system and a verifier that confirms every captured command carries a classification source and, for ML/HYBRID decisions, a real calibrated confidence. That is a live check on the deployed pipeline, not an offline metric. We can run it during the demo.

Q: What was the hardest bug you fixed?

> A strong, concrete answer -- use it: "Log rotation. Our collector tailed `cowrie.json` with a single open file handle. When Cowrie rotated the log -- renaming the old file and creating a fresh one at the same path -- our handle stayed bound to the **old inode**, so new attacks were captured by the honeypot but never ingested: the dashboard silently stopped updating while appearing healthy. We fixed it by tracking file identity as a `(device, inode)` pair and re-opening the path when that identity changes, and we switched to binary reads so byte offsets stay consistent with `os.stat().st_size`. It taught us that the dangerous failures are the silent ones -- nothing crashed, the data just stopped."

Q: What if the demo fails live?

> Have this ready: the stack is already running, and you can show the current state (sessions, events, attacker profiles) plus the PDF figures generated from `metrics.json`. Say: *"The metrics are reproducible offline from the training script, and here is the evaluation report generated directly from the artefact."*

---

10. LIMITATIONS (state these before they find them)

Volunteering limitations reads as confidence. Prepare these four:

1. **Synthetic training corpus.** Generated from curated templates, not harvested from a live production honeypot. **Quantified** by the leave-families-out score (67.87 %).
2. **Per-command classification.** We classify each command independently; a sequence-aware model could exploit inter-command context that we currently only use in the risk engine, not the classifier.
3. **Expert-assigned risk weights.** Not learned from incident outcomes -- a deliberate trade of optimality for explainability.
4. **Small support in two classes.** RECONNAISSANCE and PRIVILEGE_ESCALATION have modest support and would benefit from more data; RECONNAISSANCE is our weakest class at F1 0.714.

And the framing sentence: *"Each of these is a known trade-off with a stated mitigation path, not an unexamined gap."*

---

11. FUTURE WORK

1. **Harvest live honeypot traffic** on a public IP to close the template-generalisation gap, then re-run the identical protocol to measure whether it closed.
2. **Sequence-aware modelling** over the command stream to capture inter-command context.
3. **Learn the risk weights** from real incident outcomes while preserving an explainable additive form.
4. **Adversarial robustness** to deliberate command obfuscation (base64, variable indirection, whitespace injection).
5. **Adaptive deception** -- the highest-value direction: have the honeypot respond *differently based on inferred intent*, feeding a credential-hunting attacker fake credentials to keep them engaged and increase dwell time in the decoy.
6. **SIEM connectors** (Splunk HEC, QRadar, Sentinel) to slot into a real BFSI SOC.

---

12. TEAM & PROCESS QUESTIONS

Q: What was your individual contribution?

> Each member must have a specific, non-overlapping answer naming actual components -- e.g. "I built the hybrid classifier and the rule engine", "I did the ML pipeline: dataset generation, model selection, evaluation", "I built the async collector and persistence", "I built the React dashboard and the WebSocket layer". **Agree these before the viva** so two people do not claim the same module.

Q: How did you divide and manage the work?

> Name the split by component boundary (honeypot/collector, ML, API, frontend), and say the interfaces were defined first -- the `NormalizedEvent` model and the classifier interface -- so the modules could be built in parallel and integrated late without rework.

Q: What did you learn?

> Pick something technical and specific, not generic. Good options: that calibration is what makes confidence thresholds meaningful; that the silent failures (log rotation) are more dangerous than the loud ones; that reporting the worse number (67.87 %) makes the work more credible, not less.

Q: How long did it take / what would you do differently?

> Honest answer: harvest real traffic from day one instead of generating a corpus, because the dataset is our main limitation and everything else was built to be ready for real data anyway.

---

13. IF YOU DON'T KNOW THE ANSWER

Never bluff to an expert panel -- they will detect it and press harder. Use this:

> "That's not something we measured, so I don't want to guess. Here's how I'd find out: [state a concrete method]. What we *can* say from what we did measure is [pivot to a real number you own]."

This scores far better than a confident wrong answer. Three rules: - **Never invent a number.** If you don't remember it, say "I'd have to check the metrics file" -- every number you need is in Section 14. - **Concede small points fast**, then reframe to your strength (explainability, arbitration, honest reporting). - **If challenged on the 67.87 %**, do not get defensive. That number is *evidence of your rigour*. Say: "We could have reported only the 94 %. We report both because a single favourable split can flatter a model, and we wanted the honest picture."

---

14. NUMBERS CHEAT-SHEET (memorise before you walk in)

Quantity	Value
Held-out accuracy	**94.49 %**
Macro-F1 (headline)	**0.9229**
Macro precision / recall	0.9367 / 0.9158
Weighted F1	0.9436

Leave-families-out accuracy / macro-F1	**67.87 % / 0.5267**
Dataset	**1360** unique commands, **242** template families, **10** classes
Exact duplicates removed	5
Split	80/20 stratified -> **1088** train / **272** test
Robustness split	1055 train / 305 test
Selected model	`linsvc_calibrated` -- CalibratedClassifierCV(LinearSVC, sigmoid)
CV macro-F1 (selected)	**0.8539 +/- 0.0181** (4-fold, train split only)
Runners-up	logreg 0.8482 - complement_nb 0.8146
Best classes (F1 = 1.000)	EXECUTION, EXFILTRATION, EXPLOITATION
Weakest class	RECONNAISSANCE -- F1 **0.714**, support 16
Rule engine	**13** rules, **43** regex patterns
Thresholds	rule >= 0.90 - ML >= 0.40 - Gemini accept >= 0.85 - floor 0.10
Risk tiers	LOW 0-24 - MEDIUM 25-49 - HIGH 50-74 - CRITICAL 75-100 (alert >= 75)
Intent weights	OTHER 0 -> RECON 10 -> DISCOVERY/CLEANUP 15 -> EXECUTION 20 -> CRED_ACCESS 25 -> PRIV_ESC/PERSISTENCE/EXPLOITATION 30 -> **EXFILTRATION 35**
Reproducibility	seed **1337** - Python 3.13.12 - scikit-learn 1.9.0 - numpy 2.5.2
Ports	honeypot 2222 - API 8000 - dashboard 5173

Final checklist before the panel

- [ ] Everyone can deliver the **BFSI sector answer** (Section 1) in under a minute.
- [ ] Everyone knows **94.49 %**, **0.9229**, and **67.87 %** and what each one measures.
- [ ] Each member has a distinct, specific **individual contribution** answer.
- [ ] The stack is **already running** before you walk in -- do not build live.
- [ ] You can explain the **arbitration order** (rule -> ML -> Gemini -> fallback) from memory.
- [ ] You can say **why calibration matters** (thresholds need probabilities, not margins).
- [ ] You volunteer the **four limitations** rather than waiting to be caught.
- [ ] Nobody invents a number. Section 14 or "I'd have to check."